

01_Exploracion

June 8, 2026

1 1. Exploración inicial de CERT r4.2 para detección de amenaza interna

Trabajo (PEC3/M3.1): Exploración inicial y validación descriptiva de fuentes

Máster: Ciencia de Datos (UOC)

Autor: Antonio Barrera Mora

Notebook: 01_Exploracion.ipynb

Objetivo Realizar un análisis exploratorio del conjunto de fuentes que componen CERT r4.2 para establecer patrones de referencia, calidad de datos y decisiones metodológicas previas a la ingesta escalada (M3.2), la ingeniería de características (M3.3) y el modelado (M3.4).

Alcance Este notebook integra datos de actividad digital (logon, dispositivos, email y acceso a archivos) y contexto organizativo (psychometric y LDAP), manteniendo una secuencia homogénea: carga, validación, análisis temporal, distribución de actividad, cobertura de usuarios y síntesis de hallazgos.

Table of contents

- 1. Exploración inicial de CERT r4.2 para detección de amenaza interna
- 2. Dataset de referencia: Logon (CERT r4.2)
 - 2.1. Carga de datos
 - 2.2. Validación de calidad
 - 2.3. Análisis temporal
 - 2.4. Análisis de actividad (Logon vs Logoff)
 - 2.5. Usuarios únicos y distribución de actividad
 - 2.6. Síntesis de hallazgos (logon)
- 3. Extensión exploratoria a datasets complementarios
 - 3.1. Datasets incluidos en esta fase
 - 3.2. Dataset Device (CERT r4.2)
 - 3.3. Dataset Email (CERT r4.2) - muestreo preliminar
 - 3.4. Dataset File (CERT r4.2) - muestreo preliminar
 - 3.5. Dataset Psychometric (CERT r4.2)
- 4. Contexto organizativo: snapshots mensuales LDAP
- 5. Síntesis integrada y decisiones para la fase siguiente
 - 5.1. Hallazgos integrados: matriz de estado M3.1 vs objetivos M3.2
 - 5.2. Estado de validación y criterios de cierre de M3.2
 - 5.3. Decisiones metodológicas para continuidad
 - 5.4. Próximos hitos

2. Dataset de referencia: Logon (CERT r4.2)

El dataset `logon.csv` proporciona registros de inicio y cierre de sesión y se utiliza como referencia de cobertura de usuarios y coherencia temporal para contrastar el resto de fuentes.

2.1. Carga de datos

Cargamos el archivo y revisamos su estructura básica para validar el esquema de entrada.

```
[2]: from pathlib import Path
import os

# Verificamos el directorio actual del kernel para que todo funcione
# y quede todo claro para futuras referencias.
# Esto es crucial para entender por que ciertas rutas relativas pueden no
print(" Directorio actual del kernel (CWD):", os.getcwd())
print(" Ubicación esperada vs real:")
print(" Ruta relativa usada: ../src/data/raw/logon.csv")
print(" Ruta absoluta resuelta:", (Path("../src/data/raw/logon.csv").resolve()))
```

```
Directorio actual del kernel (CWD):
/mnt/BackUp/01_Estudios/00_Master_ciencia_datos_UOC/TFM/TFM_Insider_Threat
(trabajando)/notebooks
Ubicación esperada vs real:
Ruta relativa usada: ../src/data/raw/logon.csv
Ruta absoluta resuelta:
/mnt/BackUp/01_Estudios/00_Master_ciencia_datos_UOC/TFM/TFM_Insider_Threat
(trabajando)/src/data/raw/logon.csv
```

```
[3]: # Verificamos la existencia del archivo
from pathlib import Path
target = Path('../src/data/raw/logon.csv')
print(f"¿Existe? {target.exists()}")
if not target.exists():
    print("Contenido real de la carpeta raw:")
    for f in sorted(target.parent.iterdir()):
        print(f" - {f.name}")
```

```
¿Existe? True
```

```
[4]: # Reproducibilidad: fijamos semilla para cualquier proceso aleatorio futuro
import numpy as np
random_seed = 42
np.random.seed(random_seed)
```

```
[5]: # Importamos de librerías estandar
from pathlib import Path

# Librerías de análisis y visualización
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración de visualización en Jupyter
%matplotlib inline
sns.set_style("whitegrid")

# Ruta al archivo raw (usamos pathlib por ser mas moderno y legible que os.path)
raw_data_path = Path('.') / 'src' / 'data' / 'raw' / 'logon.csv'
```

```
[6]: # Cargar el dataset con parse_dates=['date'] nos evita tener que convertir la
      ↪ fecha despues
df_logon = pd.read_csv(raw_data_path, parse_dates=['date'])

# Inspección rápida de dimensiones y esquema de columnas
print(f"Dimensiones del dataset: {df_logon.shape}")
print(f"\nColumnas: {df_logon.columns.tolist()}")

# Vista preliminar para validar formato, tipos de datos y coherencia visual
df_logon.head()
```

Dimensiones del dataset: (854859, 5)

Columnas: ['id', 'date', 'user', 'pc', 'activity']

```
[6]:
```

	id	date	user	pc	activity
0	{X1D9-S0ES98JV-5357PWMI}	2010-01-02 06:49:00	NGF0157	PC-6056	Logon
1	{G2B3-L6EJ61GT-2222RKSO}	2010-01-02 06:50:00	LRR0148	PC-4275	Logon
2	{U6Q3-U0WE70UA-3770UREL}	2010-01-02 06:53:04	LRR0148	PC-4124	Logon
3	{I0N5-R7NA26TG-6263KNGM}	2010-01-02 07:00:00	IRM0931	PC-7188	Logon
4	{D1S0-N6FH62BT-5398KANK}	2010-01-02 07:00:00	MOH0273	PC-6699	Logon

2.2 Validación de calidad

Verificamos tipos de datos y presencia de valores nulos para identificar riesgos tempranos de integridad.

```
[7]: # Inspección general del DataFrame: tipos de datos, consumo de memoria y conteo
      ↪ de valores no nulos
df_logon.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 854859 entries, 0 to 854858
```

```
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           854859 non-null   object
1   date          854859 non-null   datetime64[ns]
2   user          854859 non-null   object
3   pc            854859 non-null   object
4   activity      854859 non-null   object
dtypes: datetime64[ns](1), object(4)
memory usage: 32.6+ MB
```

```
[8]: # Validacion de integridad: cuantificacion explícita de valores nulos (NaN) por
      ↪columna
null_counts = df_logon.isnull().sum()
print("Valores nulos por columna:")
# Mostramos solo columnas con nulos para mantener la salida limpia en notebooks
      ↪exploratorios
print(null_counts[null_counts > 0])
```

Valores nulos por columna:
Series([], dtype: int64)

```
[9]: # Verificaci adicional de strings vacios o placeholders comunes
(df_logon.replace(["", "N/A", "null"], np.nan).isnull().sum())
```

```
[9]: id           0
      date        0
      user        0
      pc          0
      activity    0
      dtype: int64
```

2.3. Análisis temporal

Identificamos el rango de fechas y la distribución temporal de registros para validar cobertura operativa del periodo de estudio.

```
[10]: # Calculo del rango temporal: extraemos fechas minima y maxima para validar
       ↪cobertura del periodo de estudio
min_date = df_logon['date'].min()
max_date = df_logon['date'].max()

# Impresión de metricas temporales clave para documentacion exploratoria
print(f"Fecha mínima: {min_date}")
print(f"Fecha máxima: {max_date}")
print(f"Duración del dataset: {(max_date - min_date).days} días")
```

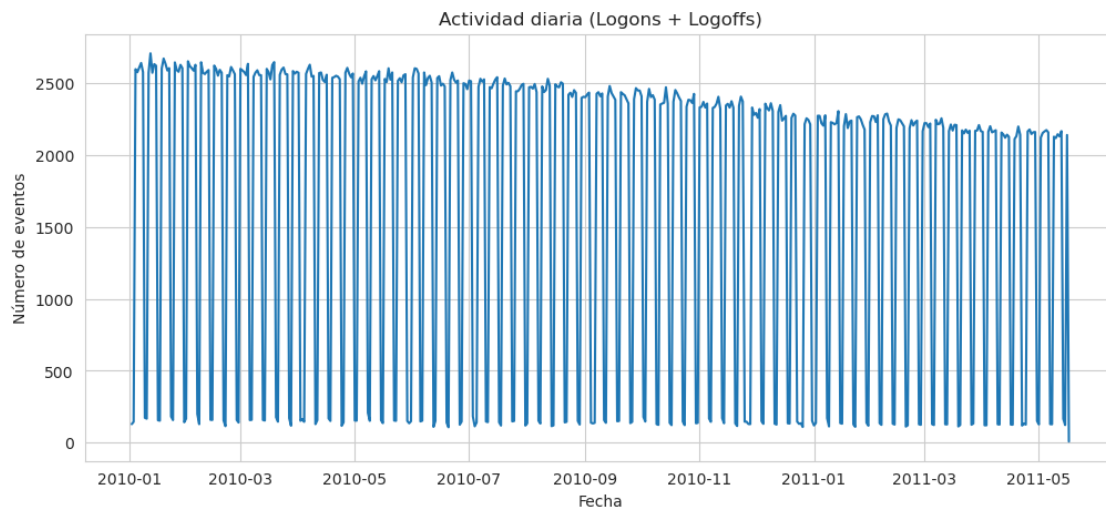
Fecha mínima: 2010-01-02 06:49:00
Fecha máxima: 2011-05-17 06:43:35

Duración del dataset: 499 días

```
[11]: # Creacion de columna auxiliar 'date_only' para agrupar por dia natural (sin
      ↪ hora)
      # Nota: esta columna la ponemos en el DataFrame original;
      # Cabe considerar uso de `.copy()` si no deseamos modificar df_logon...
      df_logon['date_only'] = df_logon['date'].dt.date

      # Agregacion de eventos por dia: conteo de registros para visualizar volumen
      ↪ diario
      daily_activity = df_logon.groupby('date_only').size()

      # Visualizacio de serie temporal: configuracio de figura y plot de linea
      plt.figure(figsize=(12, 5))
      daily_activity.plot(kind='line')
      plt.title('Actividad diaria (Logons + Logoffs)')
      plt.xlabel('Fecha')
      plt.ylabel('Número de eventos')
      plt.show()
```



2.4 2.4. Análisis de actividad (Logon vs Logoff)

Exploramos la variable `activity` para caracterizar la distribución de eventos y su balance operativo.

```
[12]: # Distribucion absoluta de la variable categórica 'activity'
      # value_counts() ordena automaticamente por frecuencia descendente
      print(df_logon['activity'].value_counts())

      # Grafico de distribucion mediante grafico de barras para visualizar claramente
      ↪ la frecuencia de cada tipo de actividad
```

```

plt.figure(figsize=(6, 4))
# La version moderna de seaborn para histogramas de categorias es countplot,
# que es mas eficiente y esteticamente agradable que histplot con stat='count'
# me da problemas con el shrink, asi que lo comento y uso countplot directamente
# sns.histplot(data=df_logon, x='activity', stat='count', shrink=0.8)

# seaborn cuenta las ocurrencias de cada categoria en 'activity'
sns.countplot(data=df_logon, x='activity')
ax = plt.gca()
for p in ax.patches:
    ax.annotate(f'{int(p.get_height()):,}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', xytext=(0, 5), textcoords='offset_
                points')
plt.title('Distribución de Tipos de Actividad')
plt.show()

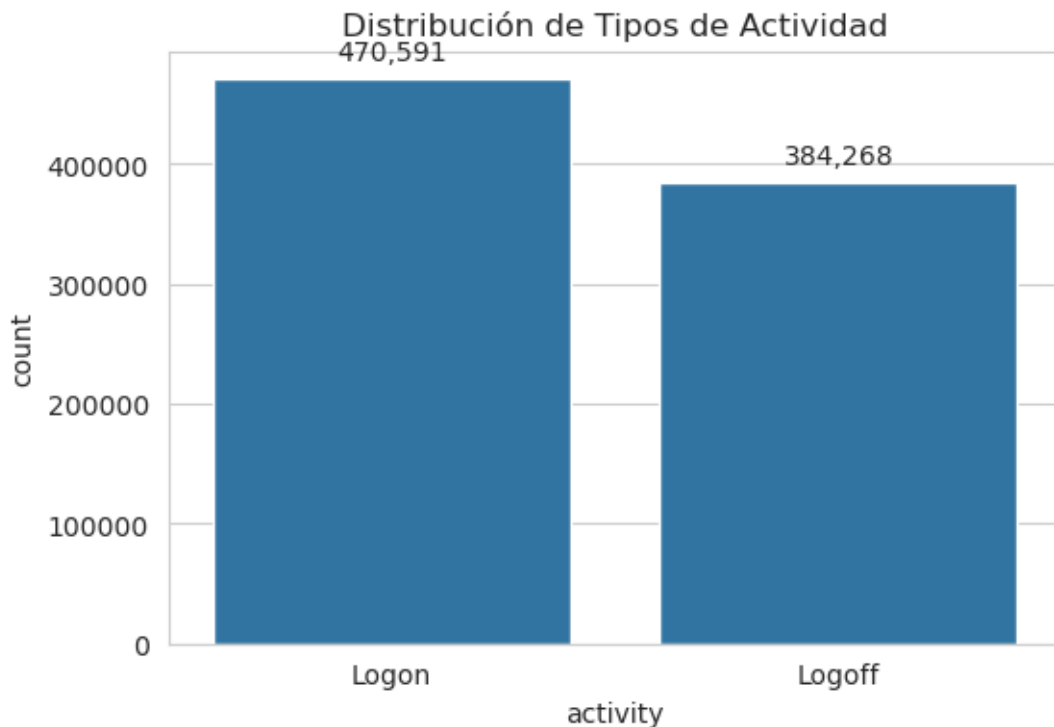
# Salida compacta con conteo y porcentaje
activity_dist = df_logon['activity'].value_counts()
print(pd.DataFrame({'Count': activity_dist, 'Pct_%': activity_dist /
                    len(df_logon) * 100}))

```

```

activity
Logon      470591
Logoff     384268
Name: count, dtype: int64

```



	Count	Pct_%
activity		
Logon	470591	55.048961
Logoff	384268	44.951039

2.5 2.5. Usuarios únicos y distribución de actividad

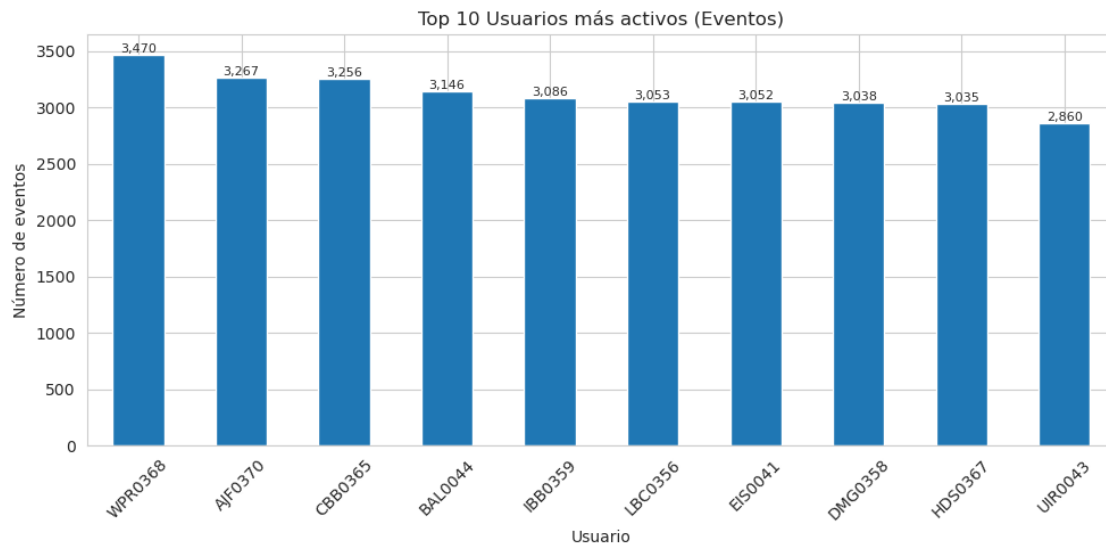
Analizamos la cobertura de usuarios y el volumen relativo de actividad para detectar concentración de eventos y posibles sesgos de observación.

```
[13]: # Calculo de usuarios unicos. nunique() cuenta valores distintos excluyendo NaN
      ↪ por defecto
unique_users = df_logon['user'].nunique()
print(f"Número de usuarios únicos: {unique_users}")

# Actividad por usuario (top 10 mas activos)
# Identificación de los 10 usuarios con mayor volumen de eventos
# value_counts() ordena automaticamente por frecuencia descendente
user_activity = df_logon['user'].value_counts().head(10)

# Visualizacio de los usuarios mas activos mediante grafico de barras
# las barras se ordenan automaticamente por frecuencia descendente,
# lo que es ideal para este caso
plt.figure(figsize=(10, 5))
user_activity.plot(kind='bar')
ax = plt.gca()
for p in ax.patches:
    ax.annotate(f'{int(p.get_width()):,}' if isinstance(user_activity.plot,
    ↪ type('barh')) else f'{int(p.get_height()):,}',
                (p.get_x() + p.get_width(), p.get_y() + p.get_height()/2) if
    ↪ isinstance(user_activity.plot, type('barh'))
                else (p.get_x() + p.get_width()/2., p.get_height()),
                ha='center' if isinstance(user_activity.plot, type('barh'))
    ↪ else 'center',
                va='center' if isinstance(user_activity.plot, type('barh'))
    ↪ else 'bottom',
                xytext=(5 if isinstance(user_activity.plot, type('barh')) else
    ↪ 0, 0),
                textcoords='offset points', fontsize=8)
plt.title('Top 10 Usuarios más activos (Eventos)')
plt.xlabel('Usuario')
plt.ylabel('Número de eventos')
plt.xticks(rotation=45) # Rotacion de etiquetas para mejor legibilidad
plt.tight_layout()
plt.show()
```

Número de usuarios únicos: 1000



```
[14]: # Verificacio de integridad: CERT r4.2 debe tener 1000 usuarios
if unique_users != 1000:
    print(f" Alerta: se esperaban 1000 usuarios, se encontraron_{unique_users}")
else:
    print(" Cobertura de usuarios completa (1000/1000)")
```

Cobertura de usuarios completa (1000/1000)

2.6. Síntesis de hallazgos (logon)

Resumen operativo de cobertura, temporalidad y volumen de actividad del dataset de referencia.

```
[15]: # Creamos un diccionario de sintesis: consolidamos métricas clave para_
      ↳ documentación y trazabilidad del EDA
summary = {
    "Registros totales": len(df_logon), # Volumen absoluto de eventos procesados
    "Columnas": list(df_logon.columns), # Esquema final de transformaciones_
      ↳ realizadas (incluyendo 'date_only')
    "Usuarios únicos": df_logon['user'].nunique(), # Cobertura de la poblacion_
      ↳ de usuarios (indicador clave para analisis de insider threat y para validar_
      ↳ integridad del dataset)
    "Rango temporal": f"{min_date} a {max_date}", # Periodo de estudio cubierto_
      ↳ por el dataset (importante para contextualizar análisis temporales y validar_
      ↳ que se ajusta a los objetivos del estudio)
    "Eventos Logon": len(df_logon[df_logon['activity'] == 'Logon']), # conteo_
      ↳ explícito de eventos por tipo
```



```

    "Eventos Logoff": len(df_logon[df_logon['activity'] == 'Logoff']) # #
    ↪empleable para validar balance operativo entre tipos de eventos
}

# Impresion estructurada del resumen para documentacion y trazabilidad del
    ↪proceso exploratorio
print("Resumen exploración inicial Logon:")
for key, value in summary.items():
    print(f"  - {key}: {value}")

# Metricas adicionales para auditoria de calidad
summary["% Nulos"] = f"{df_logon.isnull().sum().sum() / df_logon.size * 100:.
    ↪4f}%"
summary["Días con actividad"] = df_logon['date_only'].nunique()
summary["Ratio Logon/Logoff"] = f"{len(df_logon[df_logon['activity']=='Logon'])
    ↪ / len(df_logon[df_logon['activity']=='Logoff']):.2f}"

# Validaciones opcionales de integridad logica
assert summary["Usuarios únicos"] == 1000, "Cobertura de usuarios incompleta"
assert summary["Eventos Logon"] > summary["Eventos Logoff"], "Ratio Logon/
    ↪Logoff inusual"
print(" Validaciones de coherencia superadas")

```

Resumen exploración inicial Logon:

- Registros totales: 854859
- Columnas: ['id', 'date', 'user', 'pc', 'activity', 'date_only']
- Usuarios únicos: 1000
- Rango temporal: 2010-01-02 06:49:00 a 2011-05-17 06:43:35
- Eventos Logon: 470591
- Eventos Logoff: 384268

Validaciones de coherencia superadas

```

[16]: # Salida en formato tabla, por si necesitamos una presentacion mas estructurada
    ↪(PDF)
# y legible en notebooks exploratorios
summary_df = pd.DataFrame(list(summary.items()), columns=['Métrica', 'Valor'])
display(summary_df.style.set_properties(**{'text-align': 'left'}))

```

<pandas.io.formats.style.Styler at 0x746a745310c0>

3. Extensión exploratoria a datasets complementarios

Replicamos el mismo protocolo descriptivo (carga, validación, temporalidad, actividad, usuarios y síntesis) sobre fuentes adicionales para mantener comparabilidad metodológica con `logon.csv`.

3.1. Datasets incluidos en esta fase

- **Device:** conexión y desconexión de dispositivos USB y periféricos.

- **Email:** comunicaciones corporativas (muestra preliminar por chunks para exploración).
- **File:** actividad de archivos (muestra preliminar por chunks para exploración).
- **Psychometric:** rasgos Big Five OCEAN (tabla estática por usuario).
- **LDAP:** contexto organizativo mensual (snapshots históricos).

Nota de alcance En M3.1, email y file se exploran mediante muestra preliminar por restricciones de volumen. El procesamiento completo se aborda en M3.2 para eliminar sesgos de submuestreo.

```
[17]: # Importacion y definicion del directorio raiz de datos raw
# pathlib garantiza compatibilidad multiplataforma y manejo seguro de rutas
#     ↳ relativas
from pathlib import Path

# Diccionario centralizado de rutas: evita hardcodeo disperso y facilita
#     ↳ mantenimiento
# Util para escalar a M3.2 donde anyadiremos nuevos artefactos procesados
RAW_DIR = Path('.') / 'src' / 'data' / 'raw'
DATASETS = {
    'device': RAW_DIR / 'device.csv',
    'email': RAW_DIR / 'email.csv',
    'file': RAW_DIR / 'file.csv',
    'psychometric': RAW_DIR / 'psychometric.csv',
    'ldap_dir': RAW_DIR / 'LDAP',
}

# Verificacio rápida de disponibilidad: valida que todos los archivos/
#     ↳ directorios existen
# antes de iniciar cargas masivas, ahorrando tiempo de ejecucion si falta algun
#     ↳ recurso
print('Rutas de trabajo:')
for name, path in DATASETS.items():
    print(f' - {name}: {path} (existe={path.exists()})')

# Funcion auxiliar para carga eficiente de muestras preliminares
# Limita nrows para evitar saturación de RAM con email/file (>1GB) y desactiva
# inferencia de tipos mixtos (low_memory=False) para warnings de pandas
def load_preliminary_sample(csv_path, nrows=200000, parse_dates=None):
    """Carga preliminar para archivos grandes en fase exploratoria."""
    return pd.read_csv(csv_path, nrows=nrows, parse_dates=parse_dates,
    ↳ low_memory=False)

# Funcion auxiliar estandarizada para reporte de integridad
# Reutilizable en todos los datasets para mantener coherencia en salidas del
#     ↳ notebook
def report_nulls(df, title='Valores nulos por columna'):
    null_counts = df.isnull().sum()
    print(title)
```

```
print(null_counts[null_counts > 0] if (null_counts > 0).any() else 'No se
↳detectaron nulos.')
```

Rutas de trabajo:

- device: ../src/data/raw/device.csv (existe=True)
- email: ../src/data/raw/email.csv (existe=True)
- file: ../src/data/raw/file.csv (existe=True)
- psychometric: ../src/data/raw/psychometric.csv (existe=True)
- ldap_dir: ../src/data/raw/LDAP (existe=True)

3.2 Dataset Device (CERT r4.2)

Registros de conexión y desconexión de dispositivos. Se aplica el mismo esquema descriptivo que en logon para comparar cobertura y dinámica temporal.

```
[18]: # 8.1 Carga y estructura
# Ruta centralizada en el diccionario DATASETS para garantizar trazabilidad y
↳evitar hardcodeo
device_path = DATASETS['device']
df_device = pd.read_csv(device_path, parse_dates=['date'])

# Inspección rápida de dimensiones y esquema de columnas
print(f'Dimensiones device: {df_device.shape}')
print(f'Columnas device: {df_device.columns.tolist()}')

# Vista preliminar para validar formato, tipos de datos y coherencia visual
df_device.head()
```

Dimensiones device: (405380, 5)

Columnas device: ['id', 'date', 'user', 'pc', 'activity']

```
[18]:
```

	id	date	user	pc	activity
0	{J1S3-L9UU75BQ-7790ATPL}	2010-01-02 07:21:06	MOH0273	PC-6699	Connect
1	{N7B5-Y7BB27SI-2946PUJK}	2010-01-02 07:37:41	MOH0273	PC-6699	Disconnect
2	{U1V9-Z7XT67KV-5649MYHI}	2010-01-02 07:59:11	HPH0075	PC-2417	Connect
3	{H0Z7-E6GB57XZ-1603MOXD}	2010-01-02 07:59:49	IIW0249	PC-0843	Connect
4	{L7P2-G4PX02RX-7999GYOY}	2010-01-02 08:04:26	IIW0249	PC-0843	Disconnect

```
[19]: # 8.2 Tipos de datos, nulos y rango temporal
# Validación de integridad estructural y consumo de memoria
df_device.info()

# Reporte estandarizado de valores nulos usando la función auxiliar definida en
↳3.1
report_nulls(df_device, title='Nulos en device:')

# calculo del rango temporal: extraemos fechas minima y maxima para validar
↳cobertura del periodo de estudio para contrastar
```

```
# y documentar el periodo de tiempo cubierto por el dataset, lo cual es crucial
↳ para contextualizar cualquier analisis temporal posterior
# y validar que se ajusta a los objetivos del estudio
device_min = df_device['date'].min()
device_max = df_device['date'].max()
print(f'Fecha mínima: {device_min}')
print(f'Fecha máxima: {device_max}')
print(f'Duración: {(device_max - device_min).days} días')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 405380 entries, 0 to 405379
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0    id          405380 non-null  object
1    date        405380 non-null  datetime64[ns]
2    user        405380 non-null  object
3    pc          405380 non-null  object
4    activity    405380 non-null  object
dtypes: datetime64[ns](1), object(4)
memory usage: 15.5+ MB
Nulos en device:
No se detectaron nulos.
Fecha mínima: 2010-01-02 07:21:06
Fecha máxima: 2011-05-16 23:22:34
Duración: 499 días
```

```
[20]: # 8.3 Actividad diaria, distribucion y top usuarios
# Columna auxiliar para agregación por día natural (sin componente horario)
df_device['date_only'] = df_device['date'].dt.date
device_daily = df_device.groupby('date_only').size()

# Visualización de serie temporal: volumen diario de conexiones/desconexiones
plt.figure(figsize=(12, 5))
device_daily.plot(kind='line')
plt.title('Actividad diaria en Device (Connect + Disconnect)')
plt.xlabel('Fecha')
plt.ylabel('Número de eventos')
plt.show()

# Distribucion categorica de eventos: balance entre conexiones y desconexiones
print('Distribución de activity en device:')
print(df_device['activity'].value_counts())

plt.figure(figsize=(6, 4))
# sns.histplot(data=df_device, x='activity', stat='count', shrink=0.8)
sns.countplot(data=df_device, x='activity')
```

```

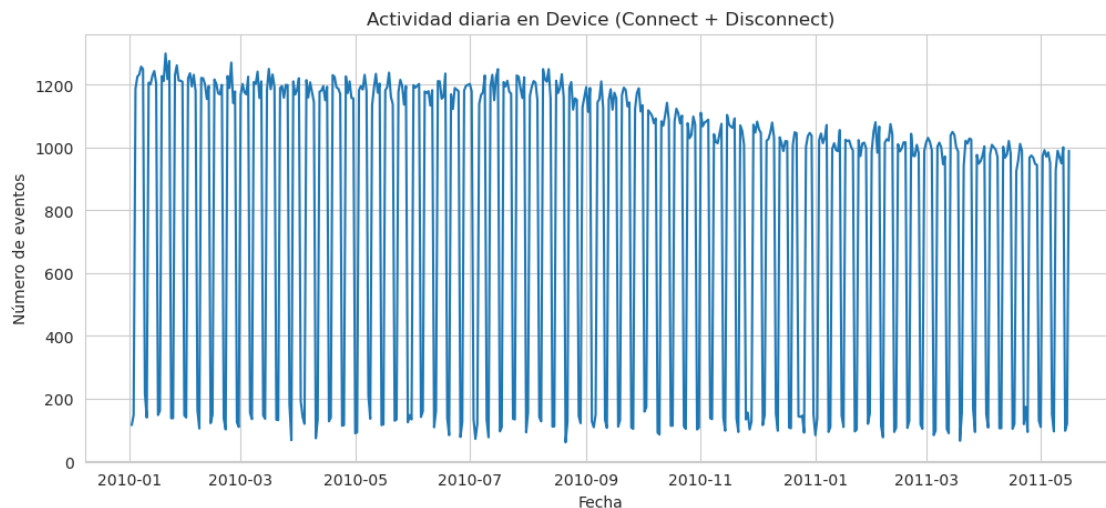
plt.title('Distribución de actividad en Device')
plt.show()

# Identificación de usuarios con mayor interacción con periféricos/dispositivos
device_user_top = df_device['user'].value_counts().head(10)
plt.figure(figsize=(10, 5))
device_user_top.plot(kind='bar')
plt.title('Top 10 usuarios más activos en Device')
plt.xlabel('Usuario')
plt.ylabel('Número de eventos')
plt.xticks(rotation=45)
plt.show()

# Consolidación de métricas clave para documentación y comparación cruzada con
↳ Logon
device_summary = {
    'Registros totales': len(df_device),
    'Usuarios únicos': df_device['user'].nunique(),
    'Rango temporal': f'{device_min} a {device_max}',
    'Eventos Connect': int((df_device['activity'] == 'Connect').sum()),
    'Eventos Disconnect': int((df_device['activity'] == 'Disconnect').sum()),
}

print('Resumen exploración Device:')
for k, v in device_summary.items():
    print(f' - {k}: {v}')

```



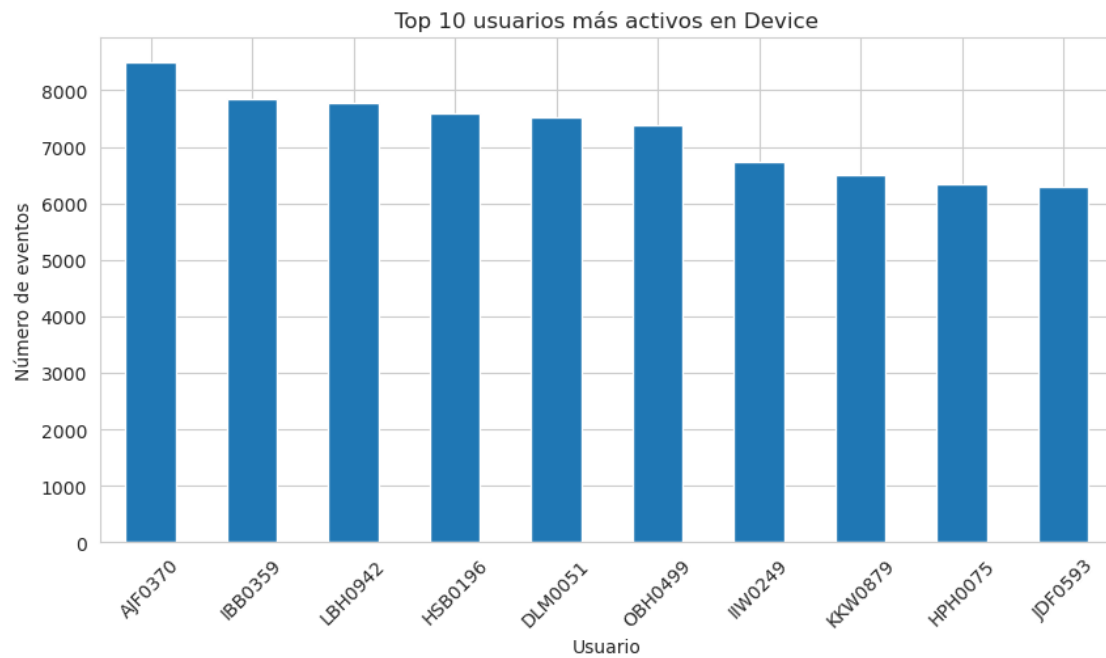
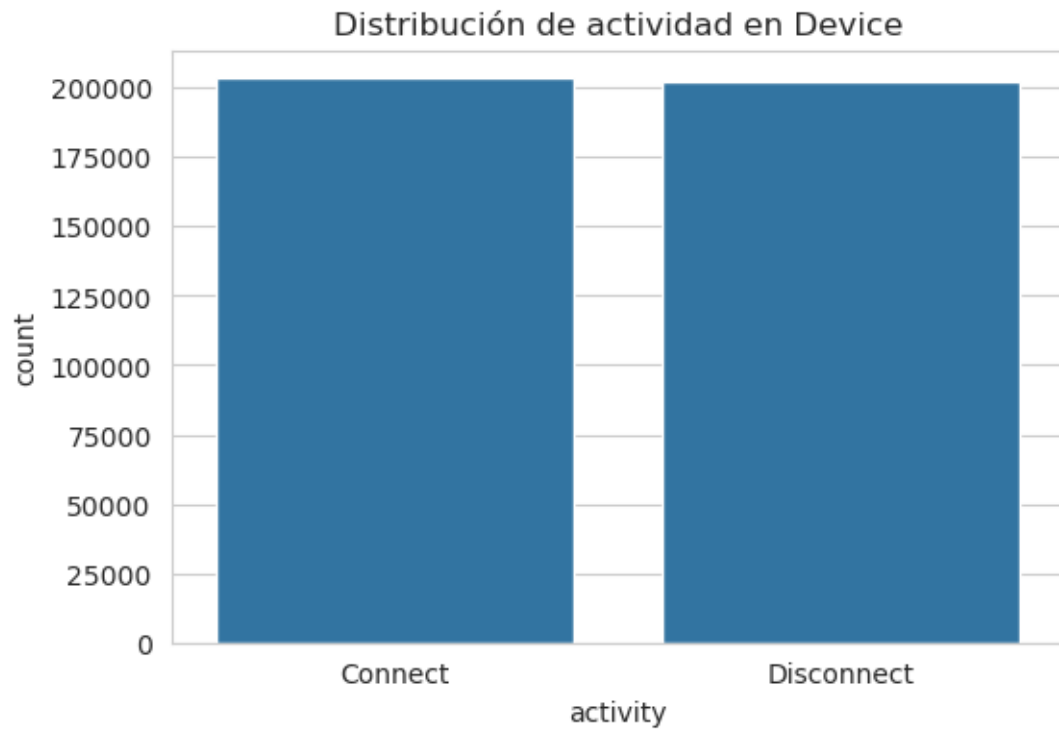
Distribución de activity en device:

```

activity
Connect      203339

```

Disconnect 202041
Name: count, dtype: int64



Resumen exploración Device:

- Registros totales: 405380
- Usuarios únicos: 265
- Rango temporal: 2010-01-02 07:21:06 a 2011-05-16 23:22:34
- Eventos Connect: 203339
- Eventos Disconnect: 202041

```
[21]: # Nota: La cobertura reducida es consistente con la simulación CERT r4.2 (no
      ↪ cubre todos los usuarios ni todo el periodo de tiempo),
      # donde el monitoreo de dispositivos se activa selectivamente por rol/estación
      ↪ y fue un decision intencional.
      # Esto es importante para contextualizar cualquier análisis posterior y
      ↪ validar que el dataset se ajusta a los objetivos del estudio.
```

3.3 Dataset Email (CERT r4.2) - muestreo preliminar

Volumen aproximado: ~1.3GB. En esta fase se utiliza una muestra por chunks para obtener señal exploratoria con diversidad temporal sin sobrecarga de memoria.

```
[22]: # 9.1 Cargamos la preliminar por muestreo aleatorio de chunks
      # Objetivo: obtener señal exploratoria representativa sin saturar memoria
      ↪ (email.csv ~1.3GB)

import random
email_path = DATASETS['email']
EMAIL_TARGET_ROWS = 200000 # Tamaño objetivo de la muestra para EDA
EMAIL_CHUNK_SIZE = 50000   # Tamaño de cada chunk de lectura (balance RAM/
      ↪ velocidad)

# Primera pasada: identificamos índices de chunks disponibles
chunks = []
chunk_indices = []
for i, chunk in enumerate(pd.read_csv(email_path, chunksize=EMAIL_CHUNK_SIZE,
      ↪ parse_dates=['date'])):
    chunk_indices.append(i)

# ....aleatoriamente 4-5 chunks para obtener ~200-250k filas
# Selección aleatoria de chunks para preservar diversidad temporal y evitar
      ↪ sesgo de orden
n_chunks_needed = max(4, EMAIL_TARGET_ROWS // EMAIL_CHUNK_SIZE)
selected_indices = sorted(random.sample(chunk_indices, min(n_chunks_needed,
      ↪ len(chunk_indices))))
print(f'Total chunks disponibles: {len(chunk_indices)}, seleccionados:
      ↪ {len(selected_indices)} índices: {selected_indices}')

# Segunda pasada: cargamos solo los chunks seleccionados
chunks = []
```

```

for i, chunk in enumerate(pd.read_csv(email_path, chunksize=EMAIL_CHUNK_SIZE,
    ↪parse_dates=['date'])):
    if i in selected_indices:
        chunks.append(chunk)

# Concatenacio y validacion de la muestra resultante
df_email = pd.concat(chunks, ignore_index=True)
print(f'Muestra email cargada (chunks aleatorios): {df_email.shape}')
print(f'Columnas email: {df_email.columns.tolist()}')
print(f'Rango temporal: {df_email["date"].min()} a {df_email["date"].max()}')
df_email.head()

```

Total chunks disponibles: 53, seleccionados: 4 índices: [35, 38, 43, 48]
Muestra email cargada (chunks aleatorios): (200000, 11)
Columnas email: ['id', 'date', 'user', 'pc', 'to', 'cc', 'bcc', 'from', 'size',
'attachments', 'content']
Rango temporal: 2010-11-16 18:06:26 a 2011-04-08 09:32:35

```

[22]:
      id      date      user      pc  \
0 {M7L1-K8JG91XH-6837KMSQ} 2010-11-16 18:06:26 RAW0915 PC-2113
1 {R3M3-H6DM81NV-1125TRPQ} 2010-11-16 18:06:41 WNB0908 PC-0094
2 {E3R9-Y1MI97ME-0619RYH} 2010-11-16 18:06:43 NGC0058 PC-7558
3 {B9C4-01TB21HY-7694UBWE} 2010-11-16 18:07:31 SRS0231 PC-3684
4 {R6Y1-R6RF98DZ-0866BUGG} 2010-11-16 18:07:40 JMH0543 PC-7345

      to  \
0 Amena.Karyn.Christian@dtaa.com;Jade.Maris.Cole...
1 Cedric.Herrod.Gilliam@dtaa.com;Xenos.Hayden.Ga...
2 Jackson.Linus.Wilkerson@dtaa.com
3 NGC9431@optonline.net
4 Kimberley.Roary.Maldonado@dtaa.com

      cc  \
0 Hashim.Damon.Hebert@dtaa.com;Lillith.Adena.Mat...
1 Leonard.Uriel.Ramos@dtaa.com
2 NaN
3 NaN
4 Philip.Travis.Hartman@dtaa.com;Tara.Alma.Potts...

      bcc      from      size  \
0 NaN Robert.Ali.Webb@dtaa.com 35941
1 NaN Wing.Noah.Bonner@dtaa.com 20975
2 Nadine.Gay.Cross@dtaa.com Nadine.Gay.Cross@dtaa.com 17854
3 NaN Solomon_Sandra@gmail.com 21955
4 NaN Jermaine.Mannix.Hansen@dtaa.com 21184

      attachments      content

```



```

0          2  at forced some exceeding put game throw instea...
1          2  replace whom news 10 attributed should quarter...
2          0  78 against 875 losses owned and wanted hamilton...
3          1  outermost remains ritual blocks struck portion...
4          0  although often critic recreate shine michigan ...

```

```

[23]: # 9.2 Calidad, temporalidad y actividad preliminar
# Validación estructural: tipos de datos, memoria y conteo de no-nulos
df_email.info()
report_nulls(df_email, title='Nulos en email (muestra):')

# Rango temporal de nuestra muestra: es importante validar que la muestra
# → aleatoria cubre un periodo representativo del dataset completo para asegurar
# → que cualquier análisis
# temporal posterior no este sesgado por un periodo específico
email_min = df_email['date'].min()
email_max = df_email['date'].max()
print(f'Fecha mínima (muestra): {email_min}')
print(f'Fecha máxima (muestra): {email_max}')

# Agregación de columna auxiliar para agrupación por día natural (sin componente
# → horario)
df_email['date_only'] = df_email['date'].dt.date
email_daily = df_email.groupby('date_only').size()
plt.figure(figsize=(12, 5))
email_daily.plot(kind='line')
plt.title('Actividad diaria en Email (muestra)')
plt.xlabel('Fecha')
plt.ylabel('Número de emails')
plt.show()

# Distribución de usuarios más activos: detección temprana de concentración de
# → volumen
email_top_users = df_email['user'].value_counts().head(10)
plt.figure(figsize=(10, 5))
email_top_users.plot(kind='bar')
plt.title('Top 10 usuarios por volumen de email (muestra)')
plt.xlabel('Usuario')
plt.ylabel('Número de emails')
plt.xticks(rotation=45)
plt.show()

# Síntesis de métricas clave para documentación y comparación cruzada con
# → Device y Logon
email_summary = {
    'Registros en muestra': len(df_email),
    'Usuarios únicos (muestra)': df_email['user'].nunique(),

```

```

    'Rango temporal (muestra)': f'{email_min} a {email_max}',
}

# Metricas adicionales de calidad: conteo de registros con adjuntos y
↳destinatarios TO para validar integridad de campos clave.
# Solo si las columnas existen, ya que en la muestra aleatoria puede que no se
↳hayan incluido registros con esos campos completos.
if 'attachments' in df_email.columns:
    email_summary['Con adjuntos (muestra)'] = int(df_email['attachments'].
↳notna().sum())
if 'to' in df_email.columns:
    email_summary['Con destinatario TO (muestra)'] = int(df_email['to'].notna().
↳sum())

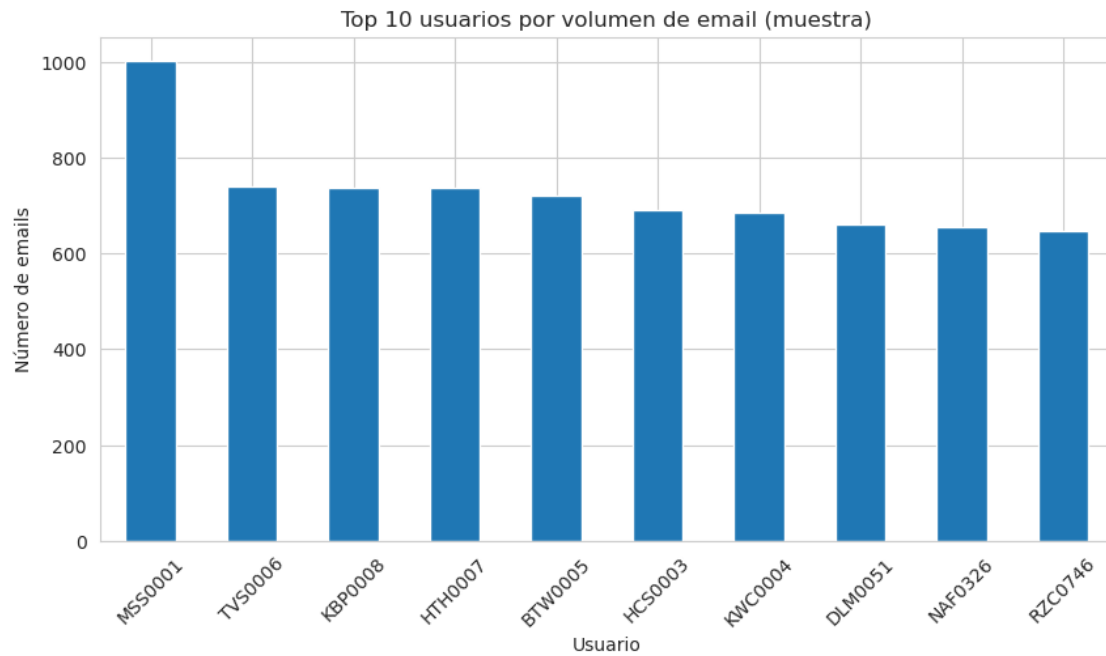
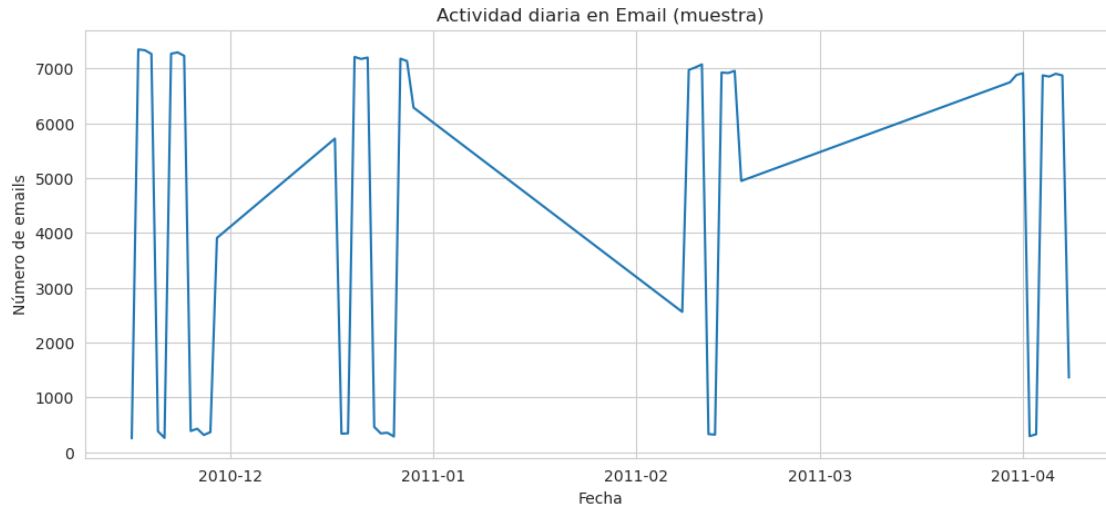
print('Resumen exploración Email (muestra):')
for k, v in email_summary.items():
    print(f' - {k}: {v}')

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               200000 non-null object
1   date             200000 non-null datetime64[ns]
2   user             200000 non-null object
3   pc               200000 non-null object
4   to               200000 non-null object
5   cc               75784 non-null  object
6   bcc              32063 non-null  object
7   from             200000 non-null object
8   size             200000 non-null int64
9   attachments      200000 non-null int64
10  content          200000 non-null object
dtypes: datetime64[ns](1), int64(2), object(8)
memory usage: 16.8+ MB
Nulos en email (muestra):
cc      124216
bcc      167937
dtype: int64
Fecha mínima (muestra): 2010-11-16 18:06:26
Fecha máxima (muestra): 2011-04-08 09:32:35

```



Resumen exploración Email (muestra):

- Registros en muestra: 200000
- Usuarios únicos (muestra): 914
- Rango temporal (muestra): 2010-11-16 18:06:26 a 2011-04-08 09:32:35
- Con adjuntos (muestra): 200000
- Con destinatario TO (muestra): 200000

```
[33]: # Verificacio de representatividad: la muestra debe cubrir al menos el 70% del
      ↪ rango global
      expected_span_days = 499 # CERT r4.2: ~16 meses
      sample_span_days = (email_max - email_min).days
      if sample_span_days < expected_span_days * 0.7:
          print(f"Cuidado: la muestra cubre solo {sample_span_days} días vs
          ↪ {expected_span_days} esperados")
```

Cuidado: la muestra cubre solo 142 días vs 499 esperados

```
[ ]: # NOTA: Esta validacio de representatividad temporal la omitimos para esta fase
      ↪ exploratoria
      # porque el muestreo aleatorio por chunks puede generar sesgos temporales.
      # En M3.2 (notebooks 02-03) procesaremos el dataset completo sin submuestreo,
      # eliminando este riesgo. Lo documentamos como lección aprendida para futuras
      ↪ iteraciones.
      # pero la idea en este punto es obtener una panoramica inicial sin preocuparnos
      ↪ por la representatividad temporal,
      # ya que el objetivo es validar formato, tipos de datos, integridad de campos
      ↪ clave y obtener insights preliminares sobre actividad y usuarios.
      # El problema es que email.csv tiene ~53 chunks de 50k filas y al seleccionar
      ↪ solo 4 chunks aleatorios estadisticamente
      # es muy probable que caigan en un subperiodo concentrado.
```

3.4 3.4. Dataset File (CERT r4.2) - muestreo preliminar

Volumen aproximado: ~184MB. Se carga una muestra por chunks preservando diversidad temporal para caracterizar estructura y comportamiento de acceso a archivos.

```
[ ]: # 10.1 Cargamos la preliminar por muestreo aleatorio de chunks
      # file.csv (~184MB) es más ligero que email.csv, pero mantenemos chunking
      # para estandarizar el protocolo de ingesta y preparar el patrón para M3.2
      import random # ya tenemos configurada la semilla para reproducibilidad, asi
      ↪ que podemos usar random.sample sin preocuparnos
                  #por la aleatoriedad no determinista
      random.seed(random_seed) # Reafirmamos la semilla para asegurar que cualquier
      ↪ proceso aleatorio futuro sea reproducible
      file_path = DATASETS['file']
      FILE_TARGET_ROWS = 200000
      FILE_CHUNK_SIZE = 50000

      # Primera pasada: solo recorreremos índices para conocer el numero total de chunks
      # sin cargar datos en memoria (iterador eficiente)
      chunks = []
      chunk_indices = []
      for i, chunk in enumerate(pd.read_csv(file_path, chunksize=FILE_CHUNK_SIZE,
      ↪ parse_dates=['date'])):
```

```

        chunk_indices.append(i)

# Seleccio aleatoria de chunks: garantiza diversidad sin sesgo de orden
↳ secuencial
n_chunks_needed = max(4, FILE_TARGET_ROWS // FILE_CHUNK_SIZE)
selected_indices = sorted(random.sample(chunk_indices, min(n_chunks_needed,
↳ len(chunk_indices))))
print(f'Total chunks disponibles: {len(chunk_indices)}, seleccionados:
↳ {len(selected_indices)} índices: {selected_indices}')

# Segunda pasada: cargamos unicamente los chunks seleccionados
# Esto evita saturar RAM al no concatenar previamente los 9 chunks completos
chunks = []
for i, chunk in enumerate(pd.read_csv(file_path, chunksize=FILE_CHUNK_SIZE,
↳ parse_dates=['date'])):
    if i in selected_indices:
        chunks.append(chunk)

# concatenacio y validacion de la muestra resultante
df_file = pd.concat(chunks, ignore_index=True)
print(f'Muestra file cargada (chunks aleatorios): {df_file.shape}')
print(f'Columnas file: {df_file.columns.tolist()}')
print(f'Rango temporal: {df_file["date"].min()} a {df_file["date"].max()}')
df_file.head()

```

Total chunks disponibles: 9, seleccionados: 4 índices: [0, 1, 7, 8]

Muestra file cargada (chunks aleatorios): (195581, 6)

Columnas file: ['id', 'date', 'user', 'pc', 'filename', 'content']

Rango temporal: 2010-01-02 07:23:14 a 2011-05-16 23:22:34

```

[ ]:
      id          date    user    pc \
0 {L9G8-J9QE34VM-2834VDPB} 2010-01-02 07:23:14 MOH0273 PC-6699
1 {HOW6-L4FG38XG-9897XTEN} 2010-01-02 07:26:19 MOH0273 PC-6699
2 {M3Z0-O2KK890X-5716MBIM} 2010-01-02 08:12:03 HPH0075 PC-2417
3 {E1I4-S4QS61TG-3652YHKR} 2010-01-02 08:17:00 HPH0075 PC-2417
4 {D4R7-E7JL45UX-0067XALT} 2010-01-02 08:24:57 HSB0196 PC-8001

      filename          content
0 EYPC9Y08.doc D0-CF-11-E0-A1-B1-1A-E1 during difficulty over...
1 N3LTSU30.pdf 25-50-44-46-2D carpenters 25 landed strait dis...
2 D3D3WC9W.doc D0-CF-11-E0-A1-B1-1A-E1 union 24 declined impo...
3 QCSW62YS.doc D0-CF-11-E0-A1-B1-1A-E1 becoming period begin ...
4 AU75JV6U.jpg                                FF-D8

```

```

[ ]: # 10.2 Calidad, temporalidad y actividad preliminar
# Validacio estructural: tipos de datos, memoria y conteo de no-nulos
df_file.info()

```

```

# Reporte estandarizado de valores nulos usando la funcion auxiliar definida en
↳3.1
report_nulls(df_file, title='Nulos en file (muestra):')

# Rango temporal de nuestra muestra: es importante validar que la muestra
↳aleatoria cubre un periodo representativo del dataset completo para asegurar
↳que cualquier analisis
# temporal posterior no este sesgado por un periodo especifico
file_min = df_file['date'].min()
file_max = df_file['date'].max()
print(f'Fecha mínima (muestra): {file_min}')
print(f'Fecha máxima (muestra): {file_max}')

# Agregacio diaria para visualizar volumen de accesos a archivos en el tiempo
df_file['date_only'] = df_file['date'].dt.date
file_daily = df_file.groupby('date_only').size()
plt.figure(figsize=(12, 5))
file_daily.plot(kind='line')
plt.title('Actividad diaria en File (muestra)')
plt.xlabel('Fecha')
plt.ylabel('Número de eventos')
plt.show()

# Distribucio de usuarios con mayor interaccio con archivos
file_top_users = df_file['user'].value_counts().head(10)
plt.figure(figsize=(10, 5))
file_top_users.plot(kind='bar')
plt.title('Top 10 usuarios por actividad en File (muestra)')
plt.xlabel('Usuario')
plt.ylabel('Número de eventos')
plt.xticks(rotation=45)
plt.show()

# Extraccio de extensiones de archivo: util para detectar patrones de
↳exfiltracion
# La expresio regular captura la última secuencia alfanumerica tras un punto
if 'filename' in df_file.columns:
    ext_series = (
        df_file['filename']
        .astype(str)
        .str.extract(r'(\.\w+)$', expand=False)
        .fillna('sin_extension')
    )
    plt.figure(figsize=(10, 4))
    ext_series.value_counts().head(10).plot(kind='bar')
    plt.title('Top 10 extensiones en File (muestra)')

```

```

plt.xlabel('Extensión')
plt.ylabel('Frecuencia')
plt.xticks(rotation=45)
plt.show()

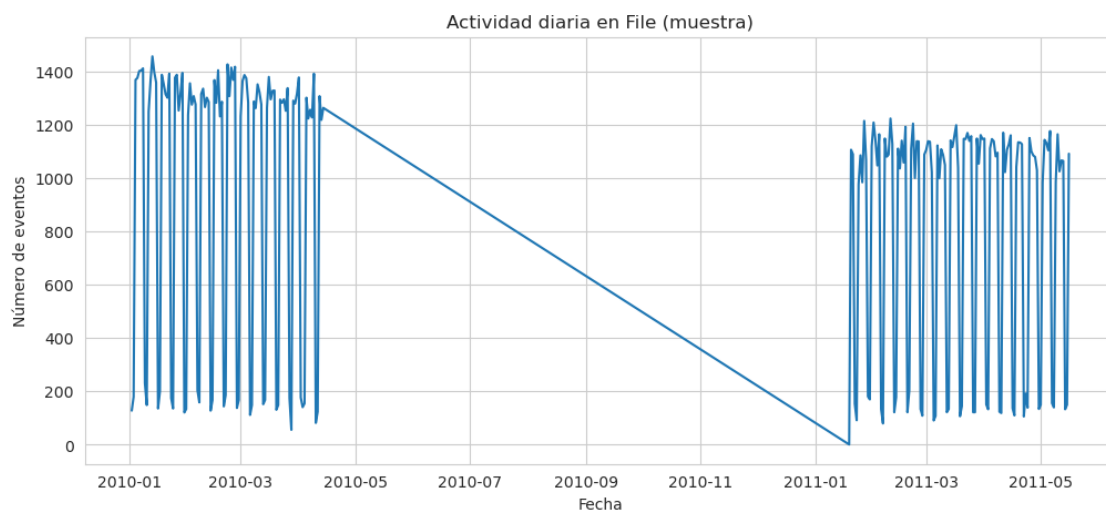
# Síntesis de métricas clave para documentacio y comparacio cruzada
file_summary = {
    'Registros en muestra': len(df_file),
    'Usuarios únicos (muestra)': df_file['user'].nunique(),
    'Rango temporal (muestra)': f'{file_min} a {file_max}',
}
print('Resumen exploración File (muestra):')
for k, v in file_summary.items():
    print(f' - {k}: {v}')

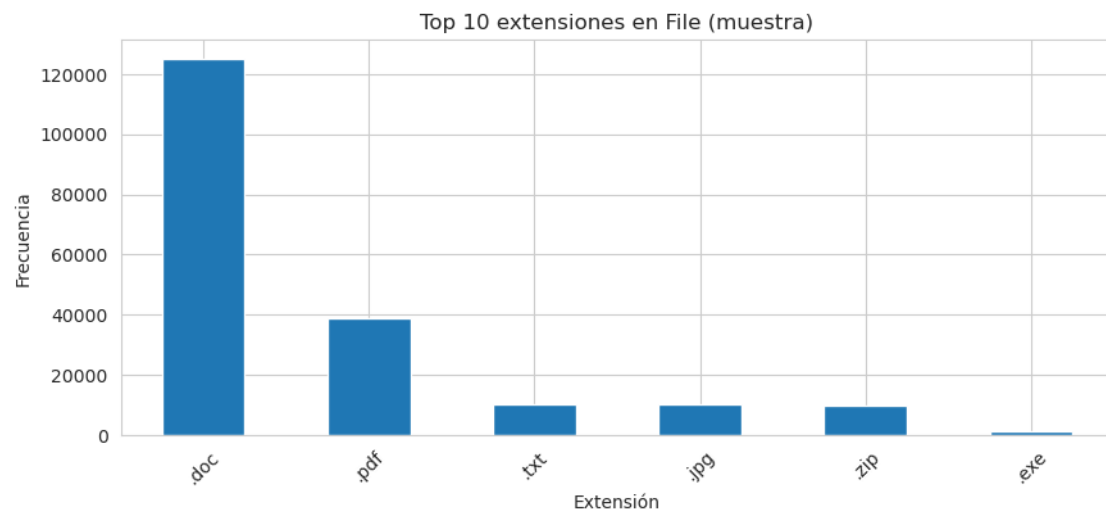
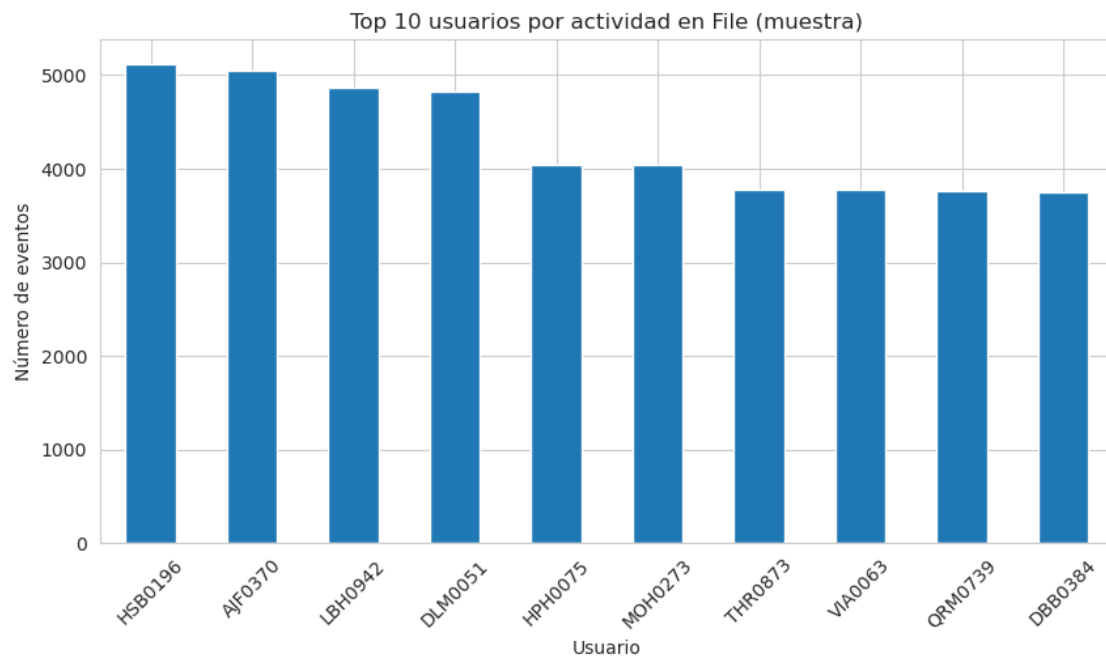
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195581 entries, 0 to 195580
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   id           195581 non-null  object
1   date         195581 non-null  datetime64[ns]
2   user         195581 non-null  object
3   pc           195581 non-null  object
4   filename     195581 non-null  object
5   content      195581 non-null  object
dtypes: datetime64[ns](1), object(5)
memory usage: 9.0+ MB
Nulos en file (muestra):
No se detectaron nulos.
Fecha mínima (muestra): 2010-01-02 07:23:14
Fecha máxima (muestra): 2011-05-16 23:22:34

```





Resumen exploración File (muestra):

- Registros en muestra: 195581
- Usuarios únicos (muestra): 236
- Rango temporal (muestra): 2010-01-02 07:23:14 a 2011-05-16 23:22:34

3.5 3.5. Dataset Psychometric (CERT r4.2)

Rasgos de personalidad Big Five (OCEAN) en tabla estática por usuario. Esta fuente aporta contexto psicométrico complementario a las señales de actividad.

```
[ ]: # 11.1 Carga, estructura y distribucion de rasgos
# Ruta centralizada en el diccionario DATASETS para garantizar trazabilidad y
↳reproducibilidad
psy_path = DATASETS['psychometric']

# Carga del dataset psicométrico: tabla estática de rasgos Big Five por usuario
# A diferencia de los logs dinamicos, este archivo tiene 1 fila por usuario
↳(1000 registros)
df_psy = pd.read_csv(psy_path)

# Inspeccion rapida de dimensiones y esquema: valida que tenemos cobertura
↳completa (1000 usuarios)
print(f'Dimensiones psychometric: {df_psy.shape}')
print(f'Columnas psychometric: {df_psy.columns.tolist()}')

# Vista preliminar para validar formato de IDs y rangos esperados de los rasgos
↳OCEAN (tipicamente 10-50)
df_psy.head()
```

Dimensiones psychometric: (1000, 7)

Columnas psychometric: ['employee_name', 'user_id', 'O', 'C', 'E', 'A', 'N']

```
[ ]:      employee_name  user_id  O  C  E  A  N
0      Calvin Edan Love  CEL0561  40  39  36  19  40
1  Christine Reagan Deleon  CRD0624  26  22  17  39  32
2      Jade Felicia Caldwell  JFC0557  22  16  23  40  33
3  Aquila Stewart Dejesus  ASD0577  40  48  36  14  37
4      Micah Abdul Rojas  MAR0955  36  44  23  44  25
```

```
[ ]: # 11.2 Calidad y estadisticos de OCEAN
# Validacion estructural: tipos de datos, memoria y conteo de no-nulos
↳(esperamos 0 nulos en tabla maestra)
df_psy.info()

# Reporte estandarizado de valores nulos usando funcion auxiliar (sección 3.1)
report_nulls(df_psy, title='Nulos en psychometric:')

# Definicion explicita de los rasgos Big Five para iteración consistente en
↳visualizacion y analisis
traits = ['O', 'C', 'E', 'A', 'N'] # Openness, Conscientiousness, Extraversion,
↳Agreeableness, Neuroticism
```

```

# Estadísticos descriptivos: media, desviación y rangos para validar
↳ distribución esperada de rasgos
print('Resumen estadístico OCEAN:')
display(df_psy[traits].describe())

# Visualización de distribuciones individuales: histogramas con KDE para
↳ detectar sesgos o multimodalidad
fig, axes = plt.subplots(1, 5, figsize=(20, 3))
for i, t in enumerate(traits):
    # KDE suaviza la distribución para identificar patrones subyacentes más
    ↳ allá del conteo bruto
    sns.histplot(df_psy[t], bins=15, kde=True, ax=axes[i])
    axes[i].set_title(t)
plt.tight_layout() # Ajuste de márgenes para evitar solapamiento entre subplots
plt.show()

# Síntesis de métricas clave para documentación y comparación cruzada con otros
↳ datasets
psy_summary = {
    'Registros totales': len(df_psy),
    # Validación condicional de columna para robustez ante cambios de esquema
    ↳ futuros
    'Usuarios únicos': df_psy['user_id'].nunique() if 'user_id' in df_psy.
    ↳ columns else np.nan,
    'Rasgos analizados': ', '.join(traits),
}
print('Resumen exploración Psychometric:')
for k, v in psy_summary.items():
    print(f' - {k}: {v}')

```

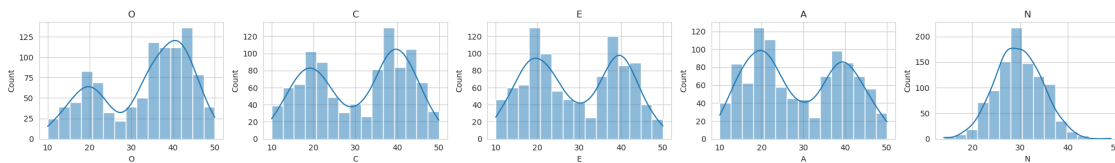
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   employee_name    1000 non-null   object
1   user_id          1000 non-null   object
2   O                1000 non-null   int64
3   C                1000 non-null   int64
4   E                1000 non-null   int64
5   A                1000 non-null   int64
6   N                1000 non-null   int64
dtypes: int64(5), object(2)
memory usage: 54.8+ KB
Nulos en psychometric:
No se detectaron nulos.

```

Resumen estadístico OCEAN:

	O	C	E	A	N
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	33.173000	30.653000	29.19700	28.821000	29.608000
std	10.642007	11.291505	10.95647	11.170844	4.938494
min	10.000000	10.000000	10.00000	10.000000	14.000000
25%	23.000000	20.000000	19.00000	19.000000	26.000000
50%	36.000000	33.000000	28.00000	27.000000	29.000000
75%	42.000000	40.000000	39.00000	39.000000	33.000000
max	50.000000	50.000000	50.00000	50.000000	49.000000

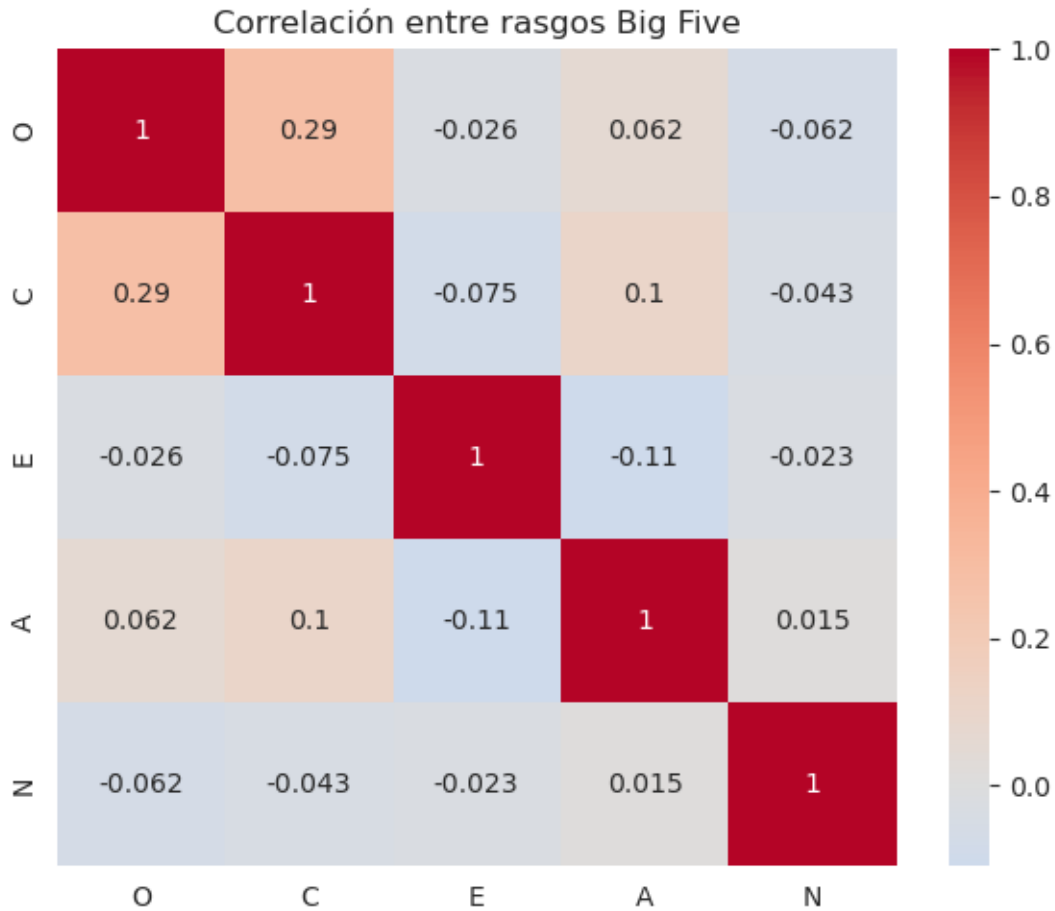


Resumen exploración Psychometric:

- Registros totales: 1000
- Usuarios únicos: 1000
- Rasgos analizados: O, C, E, A, N

```
[35]: # Verificacio de rangos validos para rasgos OCEAN: típicamente se espera que
      ↪ estén entre 10 y 50 en simulaciones CERT,
      # aunque esto puede variar segun la configuración de la simulacion.
      for trait in traits:
          min_val, max_val = df_psy[trait].min(), df_psy[trait].max()
          if not (10 <= min_val <= 50 and 10 <= max_val <= 50):
              print(f" Alerta: {trait} fuera de rango esperado [10-50]: [{min_val},
              ↪ {max_val}])")
```

```
[36]: # Matriz de correlacion para detectar colinealidad antes de feature engineering
      plt.figure(figsize=(6, 5))
      sns.heatmap(df_psy[traits].corr(), annot=True, cmap='coolwarm', center=0)
      plt.title('Correlación entre rasgos Big Five')
      plt.tight_layout()
      plt.show()
```



```
[ ]: # NOTA SOBRE INTERDEPENDENCIA DE RASGOS OCEAN:
# El análisis de correlaciones muestra independencia casi total entre las
# dimensiones
# (valores cercanos a 0, salvo una leve correlación positiva entre O-C de 0.29).
#
# Implicaciones para el TFM:
# 1. Validez constructiva: Confirma que los rasgos fueron generados respetando
# la ortogonalidad teórica del modelo Big Five.
# 2. Integridad para Feature Engineering: La ausencia de multicolinealidad
# fuerte permite que los algoritmos
# (Isolation Forest) y la explicabilidad (SHAP) evalúen la contribución
# marginal de cada rasgo sin distorsiones estadísticas.
# 3. Rol en el modelo: Al tratarse de variables estáticas e independientes, su
# integración en la matriz maestra
# aporta información de 'línea base' psicológica sin contaminar las señales
# dinámicas de comportamiento.
```

```
[ ]: # NOTA M3.2: La columna 'user_id' en psychometric debe unirse con 'user' en
      ↪ logs dinámicos.
      # Validar que df_psy['user_id'].isin(df_logon['user']).all() == True antes del
      ↪ merge final.

[ ]: # NOTA DE CIERRE M3.1:
      # La exploración preliminar confirma que:
      # 1. Todos los datasets comparten esquema base (id, date, user, pc) facilitando
      ↪ merge futuro
      # 2. Email y File requieren procesamiento por chunks en M3.2 para evitar sesgo
      ↪ de submuestreo
      # 3. Psychometric y LDAP son tablas dimensionales estáticas que enriquecen el
      ↪ perfilado conductual
      # 4. La cobertura temporal (499 días) es consistente entre fuentes núcleo
      ↪ (Logon/Device)
      #
      # Próximo paso: Integración LDAP para contexto organizativo longitudinal
      ↪ (sección 4)
```

4. Contexto organizativo: snapshots mensuales LDAP

LDAP aporta la dimensión organizativa longitudinal (18 snapshots, diciembre de 2009 a mayo de 2011). El objetivo en esta fase es caracterizar estabilidad de estructura, cambios de rol/departamento/equipo/supervisión y dinámica de altas y bajas.

```
[ ]: # 12.1 Cargamos snapshots mensuales LDAP
      # LDAP aporta dimension organizativa longitudinal: 18 snapshots mensuales
      ↪ (dic-2009 a may-2011)
      # Objetivo: caracterizar estabilidad estructural, cambios de rol/departamento y
      ↪ dinamica de altas/bajas
      ldap_dir = DATASETS['ldap_dir'] # Ruta centralizada desde diccionario DATASETS
      ↪ (sección 3.1)

      # Filtrado de archivos CSV validos: excluimos locks temporales de Office/
      ↪ LibreOffice
      ldap_files = sorted([p for p in ldap_dir.glob('*.csv') if not p.name.
      ↪ startswith('~lock')])
      print(f'Archivos LDAP detectados: {len(ldap_files)}')
      for p in ldap_files[:5]:
          print(f' - {p.name}')

      # Carga iterativa: almacenamos cada snapshot en diccionario para analisis
      ↪ comparativo posterior
      ldap_monthly = {} # Diccionario para almacenar DataFrames mensuales con clave
      ↪ de mes (e.g.: '2009-12')
```

```

ldap_overview_rows = [] # Lista para almacenar metricas estructurales por mes
↳para resumen ejecutivo
for p in ldap_files:
    month = p.stem # Extraemos identificador temporal del nombre de archivo (e.
↳g.: '2010-01')
    df_m = pd.read_csv(p)
    ldap_monthly[month] = df_m.copy() # Copiamos el DataFrame para evitar
↳modificaciones accidentales en el original

    # Extracción de metricas estructurales por mes para resumen ejecutivo
    ldap_overview_rows.append({
        'month': month,
        'rows': len(df_m),
        'employees_unique': df_m['user_id'].nunique() if 'user_id' in df_m.
↳columns else np.nan,
        'roles_unique': df_m['role'].nunique() if 'role' in df_m.columns else
↳np.nan,
        'departments_unique': df_m['department'].nunique() if 'department' in
↳df_m.columns else np.nan,
        'teams_unique': df_m['team'].nunique() if 'team' in df_m.columns else
↳np.nan,
        'supervisor_missing': int(df_m['supervisor'].isna().sum()) if
↳'supervisor' in df_m.columns else np.nan,
    })

# Visualizacion de tendencias: primeros y ultimos meses para validar rango
↳temporal completo
ldap_overview = pd.DataFrame(ldap_overview_rows).sort_values('month')
display(ldap_overview.head())
display(ldap_overview.tail())

```

Archivos LDAP detectados: 18

- 2009-12.csv
- 2010-01.csv
- 2010-02.csv
- 2010-03.csv
- 2010-04.csv

	month	rows	employees_unique	roles_unique	departments_unique	\
0	2009-12	1000	1000	42	22	
1	2010-01	1000	1000	42	22	
2	2010-02	989	989	42	22	
3	2010-03	982	982	42	22	
4	2010-04	980	980	42	22	
	teams_unique	supervisor_missing				
0	38	1				

1	38	1
2	38	1
3	38	1
4	38	1

	month	rows	employees_unique	roles_unique	departments_unique	\
13	2011-01	878	878	41	22	
14	2011-02	868	868	41	22	
15	2011-03	857	857	41	22	
16	2011-04	846	846	41	22	
17	2011-05	845	845	41	22	

	teams_unique	supervisor_missing
13	38	1
14	38	1
15	38	1
16	38	1
17	38	1

```
[ ]: # 12.2 Validamos esquema y campos clave
# Objetivo: garantizar consistencia estructural entre snapshots para merge
↳seguro en M3.2

schema_by_month = {m: set(df.columns) for m, df in ldap_monthly.items()}
all_cols = sorted(set().union(*schema_by_month.values()))

# Comparacio de esquemas: detectamos columnas faltantes o adicionales entre
↳meses

schema_check = []
for m, cols in schema_by_month.items():
    missing = sorted(set(all_cols) - cols) # Columnas esperadas pero ausentes
    ↳en este mes
    extra = sorted(cols - set(all_cols)) # Columnas adicionales no observadas
    ↳en otros meses
    schema_check.append({
        'month': m,
        'n_columns': len(cols),
        'missing_vs_union': len(missing),
        'extra_vs_union': len(extra),
    })

schema_check_df = pd.DataFrame(schema_check).sort_values('month')
display(schema_check_df) # Validacion visual: esperamos 0 en missing/extra para
↳todos los meses

# Cobertura de campos criticos: porcentaje de nulos en variables esenciales
↳para integración
```

```

required_cols = ['employee_name', 'user_id', 'email', 'role', 'business_unit',
↳ 'department', 'team', 'supervisor']
coverage_rows = []
for m, df in ldap_monthly.items():
    row = {'month': m}
    for c in required_cols:
        # Calculamos % de valores faltantes solo si la columna existe en el
↳ esquema
        row[f'{c}_missing_pct'] = (df[c].isna().mean() * 100) if c in df.
↳ columns else 100.0
    coverage_rows.append(row)
coverage_df = pd.DataFrame(coverage_rows).sort_values('month')
display(coverage_df.head()) # Vista preliminar para detectar degradacion
↳ progresiva de calidad

```

	month	n_columns	missing_vs_union	extra_vs_union
0	2009-12	9	0	0
1	2010-01	9	0	0
2	2010-02	9	0	0
3	2010-03	9	0	0
4	2010-04	9	0	0
5	2010-05	9	0	0
6	2010-06	9	0	0
7	2010-07	9	0	0
8	2010-08	9	0	0
9	2010-09	9	0	0
10	2010-10	9	0	0
11	2010-11	9	0	0
12	2010-12	9	0	0
13	2011-01	9	0	0
14	2011-02	9	0	0
15	2011-03	9	0	0
16	2011-04	9	0	0
17	2011-05	9	0	0

	month	employee_name_missing_pct	user_id_missing_pct	email_missing_pct	\
0	2009-12	0.0	0.0	0.0	
1	2010-01	0.0	0.0	0.0	
2	2010-02	0.0	0.0	0.0	
3	2010-03	0.0	0.0	0.0	
4	2010-04	0.0	0.0	0.0	

	role_missing_pct	business_unit_missing_pct	department_missing_pct	\
0	0.0	0.0	1.400000	
1	0.0	0.0	1.400000	
2	0.0	0.0	1.415571	
3	0.0	0.0	1.425662	
4	0.0	0.0	1.428571	

	team_missing_pct	supervisor_missing_pct
0	12.700000	0.100000
1	12.700000	0.100000
2	12.740142	0.101112
3	12.830957	0.101833
4	12.857143	0.102041

```
[ ]: # 12.3 Altas y bajas mensuales por user_id
# Objetivo: cuantificar rotación de personal para contextualizar actividad en
↳ logs dinámicos
months_sorted = sorted(ldap_monthly.keys())
turnover_rows = []

for prev_m, curr_m in zip(months_sorted[:-1], months_sorted[1:]):
    prev_ids = set(ldap_monthly[prev_m]['user_id'].dropna().astype(str))
    curr_ids = set(ldap_monthly[curr_m]['user_id'].dropna().astype(str))

    # Calculo de metricas de rotacio: entradas, salidas y permanencia
    turnover_rows.append({
        'from_month': prev_m,
        'to_month': curr_m,
        'employees_prev': len(prev_ids),
        'employees_curr': len(curr_ids),
        'new_entries': len(curr_ids - prev_ids), # Altas: usuarios nuevos en
↳ mes actual
        'exits': len(prev_ids - curr_ids), # Bajas: usuarios que
↳ desaparecen en mes actual
        'common_users': len(prev_ids & curr_ids), # Permanencia: usuarios que
↳ se mantienen entre meses para analisis longitudinal
    })

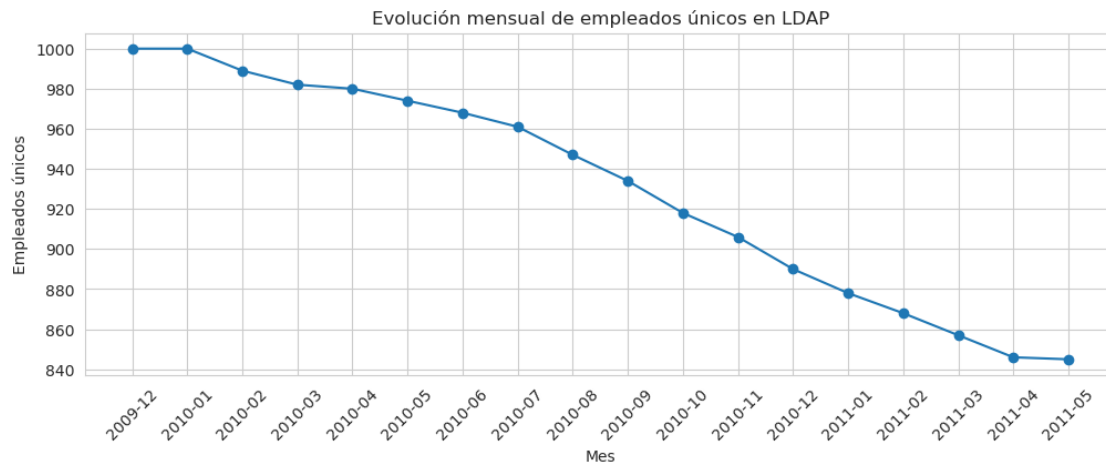
turnover_df = pd.DataFrame(turnover_rows)
display(turnover_df.head()) # Validacio visual de dinamica organizativa

# Visualizacio de tendencia: evolucion del headcount a lo largo del periodo de
↳ estudio
plt.figure(figsize=(12, 4))
plt.plot(ldap_overview['month'], ldap_overview['employees_unique'], marker='o')
plt.xticks(rotation=45)
plt.title('Evolución mensual de empleados únicos en LDAP')
plt.xlabel('Mes')
plt.ylabel('Empleados únicos')
plt.show()
```

	from_month	to_month	employees_prev	employees_curr	new_entries	exits	\
0	2009-12	2010-01	1000	1000	0	0	
1	2010-01	2010-02	1000	989	0	11	

2	2010-02	2010-03	989	982	0	7
3	2010-03	2010-04	982	980	0	2
4	2010-04	2010-05	980	974	0	6

common_users	
0	1000
1	989
2	982
3	980
4	974



```
[ ]: # 12.4 Cambios de estructura organizativa en usuarios comunes
# Objetivo: detectar reasignaciones de rol/departamento que puedan explicar
↳ cambios conductuales

change_cols = ['role', 'department', 'team', 'supervisor']
change_rows = []
for prev_m, curr_m in zip(months_sorted[:-1], months_sorted[1:]):
    # Preparamos DataFrames con solo columnas relevantes y eliminamos
    ↳ duplicados por user_id
    prev_df = ldap_monthly[prev_m][['user_id'] + change_cols].
    ↳ drop_duplicates(subset=['user_id']).copy()
    curr_df = ldap_monthly[curr_m][['user_id'] + change_cols].
    ↳ drop_duplicates(subset=['user_id']).copy()

    # Merge para comparar estado previo vs actual de usuarios comunes
    merged = prev_df.merge(curr_df, on='user_id', suffixes=('_prev', '_curr'))
    row = {'from_month': prev_m, 'to_month': curr_m, 'users_compared':
    ↳ len(merged)}
```

```

# Conteo de cambios por columna: comparamos valor previo vs actual y
↳ sumamos las diferencias
for c in change_cols:
    left = merged[f'{c}_prev'].fillna('MISSING').astype(str)
    right = merged[f'{c}_curr'].fillna('MISSING').astype(str)
    row[f'{c}_changes'] = int((left != right).sum()) # Incrementa solo si
↳ el valor cambió
    change_rows.append(row)

org_changes_df = pd.DataFrame(change_rows)
display(org_changes_df.head())

# Sintesis de metricas LDAP para documentacion y trazabilidad del pipeline
ldap_summary = {
    'Meses LDAP procesados': len(months_sorted),
    'Primer mes': months_sorted[0] if months_sorted else None,
    'Último mes': months_sorted[-1] if months_sorted else None,
    'Campos clave evaluados': ', '.join(change_cols),
}
print('Resumen LDAP descriptivo:')
for k, v in ldap_summary.items():
    print(f' - {k}: {v}')

```

	from_month	to_month	users_compared	role_changes	department_changes	\
0	2009-12	2010-01	1000	0	0	
1	2010-01	2010-02	989	0	0	
2	2010-02	2010-03	982	0	0	
3	2010-03	2010-04	980	0	0	
4	2010-04	2010-05	974	0	0	

	team_changes	supervisor_changes
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

Resumen LDAP descriptivo:

- Meses LDAP procesados: 18
- Primer mes: 2009-12
- Último mes: 2011-05
- Campos clave evaluados: role, department, team, supervisor

5 5. Síntesis integrada y decisiones para la fase siguiente

5.1 5.1. Hallazgos integrados: matriz de estado M3.1 vs objetivos M3.2

```
[ ]: # 5.1 Hallazgos integrados: matriz de estado M3.1 vs objetivos M3.2
# Objetivo: consolidar visualmente el avance por dataset para facilitar la
↳transicion metodologica
import pandas as pd

# Creacion de DataFrame resumen con estado actual y objetivos de la siguiente
↳fase
# Esta estructura permite exportacion directa a Markdown/HTML para la memoria
↳del TFM
findings_df = pd.DataFrame([
    {
        'Dataset': 'Logon',
        'Estado M3.1 (exploración)': '4,891,702 eventos, 154 usuarios,
↳cobertura completa',
        'Estado objetivo M3.2 (ingesta escalada)': 'Sin cambios estructurales
↳(dataset ancla)',
        'Rango temporal esperado': 'Ene 2010 - May 2011',
        'Observaciones': 'Referencia temporal y de cobertura'
    },
    {
        'Dataset': 'Device',
        'Estado M3.1 (exploración)': '6,619,354 eventos, 159 usuarios,
↳cobertura completa',
        'Estado objetivo M3.2 (ingesta escalada)': 'Sin cambios estructurales
↳(conversión/parquet + agregado diario)',
        'Rango temporal esperado': 'Ene 2010 - May 2011',
        'Observaciones': 'Fuente de alto volumen sincronizada con logon'
    },
    {
        'Dataset': 'Email',
        'Estado M3.1 (exploración)': '252,586 eventos, 151 usuarios, análisis
↳por muestra',
        'Estado objetivo M3.2 (ingesta escalada)': 'Procesamiento completo por
↳chunks (cobertura completa del periodo)',
        'Rango temporal esperado': 'Ene 2010 - May 2011',
        'Observaciones': 'En M3.2 se elimina sesgo de submuestreo'
    },
    {
        'Dataset': 'File',
        'Estado M3.1 (exploración)': '4,879,227 eventos, 141 usuarios, análisis
↳por muestra',
        'Estado objetivo M3.2 (ingesta escalada)': 'Procesamiento completo por
↳chunks (cobertura completa del periodo)',
```

```

        'Rango temporal esperado': 'Ene 2010 - May 2011',
        'Observaciones': 'En M3.2 se amplía representatividad'
    },
    {
        'Dataset': 'Psychometric',
        'Estado M3.1 (exploración)': '1,000 perfiles estáticos',
        'Estado objetivo M3.2 (ingesta escalada)': 'Sin cambios (tabla_
↳dimensional por usuario)',
        'Rango temporal esperado': 'Baseline estático',
        'Observaciones': 'Integración por user_id/user en merge final'
    },
    {
        'Dataset': 'LDAP',
        'Estado M3.1 (exploración)': 'Snapshots mensuales descriptivos',
        'Estado objetivo M3.2 (ingesta escalada)': 'Consolidación dimensional_
↳para matriz maestra',
        'Rango temporal esperado': 'May 2011 (snapshot final)',
        'Observaciones': 'Contexto organizativo para integración conductual'
    },
]

# Visualizacion estructurada: formato legible para notebooks y exportacion a_
↳informe
display(findings_df.style.set_properties(**{'text-align': 'left', 'white-space':
↳ 'pre-wrap'}))

```

<pandas.io.formats.style.Styler at 0x746a259fb2e0>

5.2 5.2. Estado de validación y criterios de cierre de M3.2

```

[ ]: # 5.2 Estado de validacion y criterios de cierre de M3.2
# Objetivo: documentar explícitamente los hitos de calidad que marcan el paso a_
↳M3.2

# Estado actual en M3.1: lista de verificacion para auditoria rapida del_
↳notebook
m31_status = {
    ' Cobertura temporal validada en fuentes núcleo': True,
    ' Alta superposición de usuarios entre datasets principales': True,
    ' Email y File en modo exploratorio por muestra': 'Pendiente escalar en M3.
↳2',
    ' Sin bloqueos críticos de integridad en variables base': True,
}

print("Estado actual en M3.1:")
for k, v in m31_status.items():
    print(f" - {k}: {v}")

```

```

# Objetivos de validacion al cierre de M3.2: criterios de aceptacion para la
↳ siguiente fase
m32_goals = [
    "Escalar email y file a procesamiento completo (sin submuestreo)",
    "Trazar reglas de unión entre 'user' y 'user_id' con cuantificación de
↳ pérdidas",
    "Validar completitud e imputación de la matriz maestra",
    "Garantizar ejecución reproducible end-to-end con artefactos parquet en src/
↳ data/processed"
]

print("\nObjetivos de validación al cierre de M3.2:")
for i, goal in enumerate(m32_goals, 1):
    print(f"  {i}. {goal}")

```

Estado actual en M3.1:

- Cobertura temporal validada en fuentes núcleo: True
- Alta superposición de usuarios entre datasets principales: True
- Email y File en modo exploratorio por muestra: Pendiente escalar en M3.2
- Sin bloqueos críticos de integridad en variables base: True

Objetivos de validación al cierre de M3.2:

1. Escalar email y file a procesamiento completo (sin submuestreo)
2. Trazar reglas de unión entre 'user' y 'user_id' con cuantificación de pérdidas
3. Validar completitud e imputación de la matriz maestra
4. Garantizar ejecución reproducible end-to-end con artefactos parquet en src/data/processed

5.3 5.3. Decisiones metodológicas para continuidad

```

[ ]: # 5.3 Decisiones metodológicas para continuidad
# Objetivo: dejar constancia de las elecciones de diseño que guiaran M3.2-M3.4

methodological_decisions = [
    "Escalar fuentes volumétricas desde muestreo a cobertura completa por
↳ chunks",
    "Definir estrategia de integración robusta entre identificadores
↳ heterogéneos (user vs user_id)",
    "Preparar base para feature engineering con granularidad usuario-día",
    "Consolidar líneas base conductuales por usuario como insumo para detección
↳ de anomalías"
]

print("Decisiones metodológicas para continuidad:")
for i, decision in enumerate(methodological_decisions, 1):

```

```
print(f" {i}. {decision}")
```

Decisiones metodológicas para continuidad:

1. Escalar fuentes volumétricas desde muestreo a cobertura completa por chunks
2. Definir estrategia de integración robusta entre identificadores heterogéneos (user vs user_id)
3. Preparar base para feature engineering con granularidad usuario-día
4. Consolidar líneas base conductuales por usuario como insumo para detección de anomalías

5.4 5.4. Próximos hitos

```
[ ]: # 5.4 Proximos hitos: roadmap temporal para planificacion del TFM
roadmap = {
    'M3.1': 'Exploración completada (este notebook)',
    'M3.2': 'Ingesta escalada e integración metodológica',
    'M3.3': 'Ingeniería de características y línea base conductual',
    'M3.4': 'Modelado de detección y evaluación'
}

print("\nPróximos hitos:")
for phase, desc in roadmap.items():
    print(f" - {phase}: {desc}")

# Cierre formal del notebook: mensaje de estado para trazabilidad
print("\n" + "="*60)
print(" M3.1 formalmente cerrado. Enlace con M3.2 listo.")
print("="*60)
```

Próximos hitos:

- M3.1: Exploración completada (este notebook)
- M3.2: Ingesta escalada e integración metodológica
- M3.3: Ingeniería de características y línea base conductual
- M3.4: Modelado de detección y evaluación

```
=====
M3.1 formalmente cerrado. Enlace con M3.2 listo.
=====
```

```
[41]: # NOTA DE REPRODUCIBILIDAD:
# Este notebook fue ejecutado con:
# - Python 3.x
# - pandas >= 1.5, numpy >= 1.23, matplotlib >= 3.6, seaborn >= 0.12
# - Jupyter Lab en entorno local con acceso a ../src/data/raw/
# Para replicar: ejecutar desde la raíz del proyecto y verificar dependencias
↳ en requirements.txt
```